Control Flow and Exception Handling

Asst.Prof. Dr. Umaporn Supasitthimethee
ผศ.ดร.อุมาพร สุภสิทธิเมธี

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Error/Error
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Control_flow_and_error_handling#exception_handling_statements

Error Class and Constructor

- The **Error()** constructor creates Error objects.

```
//new Constructor
new Error()
new Error(message)

//function call - When Error is used like a function, that is without new, it will return an Error object.
Error()
Error(message)
```

```
const y = new Error('I was constructed via the "new" keyword!')
const x = Error("I was created using a function call!")
```

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Error/Error

Error Subclass

- Error objects are a special type of fundamental object. They include the basic Error type, as well as several specialized error types.

Error	คำอธิบาย	ตัวอย่าง
Error	คลาสพื้นฐานของ error ทั้งหมดใน JavaScript ใช้สร้าง error ทั่วไป	`throw new Error('Something went wrong')`
RangeError	เกิดเมื่อค่าที่ส่งเข้าไปอยู่นอกช่วงที่ยอมรับได้	`const arr = new Array(-1)`
ReferenceError	เกิดเมื่อเรียกใช้ตัวแปรที่ไม่มีอยู่	`console.log(myVar)` `// ReferenceError: myVar is not defined`
SyntaxError	เกิดเมื่อเขียน syntax ผิด	`if (true { console.log('hello')}`
TypeError	เกิดเมื่อชนิดข้อมูลไม่ถูกต้องกับ operation นั้น	`const n = 5` `n.toUpperCase()`
URIError	เกิดจากการใช้ฟังก์ชันเกี่ยวกับ URI ผิดรูปแบบ	`decodeURIComponent('%')`

InternalError	Error ภายใน engine ของ JavaScript บาง browser เช่น recursion มากเกินไป	`function loop() {`
		`  loop()`
		`}`
		`loop()`
		`// InternalError หรือ RangeError`

What is an Exception in JavaScript?

- An **Exception** is an error that happens **while your program is running**, **for example:**
  - Using a variable that doesn’t exist
  - Calling a function incorrectly
  - Getting bad data from an API
  - User input that causes a crash

Exception

- In JavaScript, an **exception** is an event that disrupts the normal flow of program execution, usually due to an error, which causes the program to stop unless handled.
- The mechanism for managing these events is called **exception handling**, which uses try...catch...finally statements to gracefully manage errors.

```
try {
  const err = new Error("Invalid amount")
// create Error
  throw err // cause exception happen
} catch (e) {
  console.log(e)
}
console.log('end working')
```

```
//output
Error: Invalid amount
   at Object.<anonymous> (C:\driveD\Working\2-
2568\INT141\SourceCodes\11\test.js:2:15)
   at Module._compile (node:internal/modules/cjs/loader:1761:14)
   at Object..js (node:internal/modules/cjs/loader:1893:10)
   at Module.load (node:internal/modules/cjs/loader:1481:32)
   at Module._load (node:internal/modules/cjs/loader:1300:12)
   at TracingChannel.traceSync (node:diagnostics_channel:328:14)
   at wrapModuleLoad (node:internal/modules/cjs/loader:245:24)
   at Module.executeUserEntryPoint [as runMain]
(node:internal/modules/run_main:154:5)
   at node:internal/main/run_main_module:33:47
end working
```

try…catch Statement

- The ***try...catch*** statement consists of a ***try*** block, which contains one or more statements, and a ***catch*** block, containing statements that specify what to do if an exception is thrown in the ***try*** block.
- In other words, you want the ***try*** block to succeed—but if it does not, you want control to pass to the ***catch*** block.
- If any statement within the ***try*** block (or in a function called from within the ***try*** block) throws an exception, control immediately shifts to the ***catch*** block.
- If no exception is thrown in the ***try*** block, the ***catch*** block is skipped.
- The ***finally*** block executes after the ***try*** and ***catch*** blocks execute but before the statements following the ***try...catch*** statement.

Key Mechanisms

keyword	Description
try	Defines a block of code to be tested for errors while it is being executed.
catch	Contains the code that executes if an error occurs within the try block. It takes an argument (the exception/error object) that provides information about the error.
finally	Contains code that executes after both the try and catch blocks, regardless of whether an exception was thrown or caught. It is useful for cleanup operations, like closing files or releasing resources.
throw	Used to create and trigger a custom error (exception) manually. The execution of the current function will stop, and control will be passed to the first enclosing catch block.

try…catch…finally

```
try {
  // code that might cause an error
} catch (error) {
  // code that runs if an error happens
}
finally{
//code that executes after both the try and catch blocks, regardless of
whether an exception was thrown or caught.
}
```

try…catch with Error object

- Inside catch, JavaScript gives you an **error object**.
- Common properties:
  - **error.name**: type of error (TypeError, ReferenceError, etc.)
  - **error.message**: human-readable message
  - **error.stack:** provides a string representation of the call stack at the time an error was created.

```
try {
  x()
} catch (error) {
  console.log(error.name)
  console.log(error.message)
  console.log(error.stack)
}
```

```
//output
ReferenceError
x is not defined
ReferenceError: x is not defined
    at Object.<anonymous> (C:\driveD\Working\2-
2568\INT141\SourceCodes\11\test.js:2:3)
    at Module._compile (node:internal/modules/cjs/loader:1761:14)
    at Object..js (node:internal/modules/cjs/loader:1893:10)
    at Module.load (node:internal/modules/cjs/loader:1481:32)
    at Module._load (node:internal/modules/cjs/loader:1300:12)
    at TracingChannel.traceSync (node:diagnostics_channel:328:14)
    at wrapModuleLoad (node:internal/modules/cjs/loader:245:24)
    at Module.executeUserEntryPoint [as runMain]
(node:internal/modules/run_main:154:5)
    at node:internal/main/run_main_module:33:47
```

try…catch with Error object

```
try {
  const num = Number('abc') //TypeError
  // const num = Number(-2) //RangeError
  if (Number.isNaN(num)) {
    throw new TypeError('Not a number')
  }
  if (num < 0) {
    throw new RangeError('Negative number not allowed')
  }
}
catch (e) {
  if (e instanceof TypeError) {
    console.log('Type Error:', e.message)
  } else if (e instanceof RangeError) {
    console.log('Range Error:', e.message)
  } else {
    console.log('Unknown Error:', e.message)
  }
}
```

```
// handle with function

try {
  // throw new RangeError('Out of range')
  throw new TypeError('Not a number')
} catch (e) {
  if (e instanceof TypeError) {
    handleTypeError(e)
  } else if (e instanceof RangeError) {
    handleRangeError(e)
  }
}
function handleTypeError(e) {
  console.log('Type Error:', e.message)
}

function handleRangeError(e) {
  console.log('Range Error:', e.message)
}
```

Using finally (optional but useful)

- finally **always runs**, whether an error happens or not.

```
try {
  console.log('Start')
  throw new Error('Oops!')
} catch (e) {
  console.log(`Error caught: ${e.message}`)
} finally {
  console.log('Cleanup here')
}

//output
Start
Error caught: Oops!
Cleanup here
```

try…catch…finally Example

- It is also important to note that the finally block will execute whether or not an exception is thrown. If an exception is thrown, however, the statements in the finally block execute even if no catch block handles the exception that was thrown.
- If an exception is thrown while the file is open, the finally block closes the file before the script fails. Using finally here ensures that the file is never left open, even if an error occurs.

```
openMyFile()
try {
  writeMyFile(theData) // This may throw an error
} catch (e) {
  handleError(e) // If an error occurred, handle it
} finally {
  closeMyFile() // Always close the resource
}
```

Be careful

If the finally block returns a value, this value becomes the return value of the entire try...catch...finally production, regardless of any return statements in the try and catch blocks:

```
function f() {
  try {
    console.log(0)
    throw "bogus"
  } catch (e) {
    console.log(1)
    // This return statement is suspended
    // until finally block has completed
    return true
    console.log(2) // not reachable
  } finally {
    console.log(3)
    return false // overwrites the previous "return"
    // `f` exits here
    console.log(4) // not reachable
  }
  console.log(5) // not reachable
}
console.log(f()) // 0, 1, 3, false
```

throw Statement with Error object

You can throw exceptions using the ***throw*** statement and handle them using ***try...catch*** statements.

```
function withdraw(amount) {
  if (amount <= 0) {
    throw new Error("Amount must be greater
than zero")
  }
  console.log("Withdrawing:", amount)
}

try {
  withdraw(-10)
} catch (e) {
  console.log(e.name)
  console.log(e.message)
  console.log(e.stack)
}
```

```
//output
Error
Amount must be greater than zero
Error: Amount must be greater than zero
    at withdraw (C:\driveD\Working\2-
2568\INT141\SourceCodes\11\test.js:3:11)
    at Object.<anonymous> (C:\driveD\Working\2-
2568\INT141\SourceCodes\11\test.js:9:3)
    at Module._compile (node:internal/modules/cjs/loader:1761:14)
    at Object..js (node:internal/modules/cjs/loader:1893:10)
    at Module.load (node:internal/modules/cjs/loader:1481:32)
    at Module._load (node:internal/modules/cjs/loader:1300:12)
    at TracingChannel.traceSync (node:diagnostics_channel:328:14)
    at wrapModuleLoad (node:internal/modules/cjs/loader:245:24)
    at Module.executeUserEntryPoint [as runMain]
(node:internal/modules/run_main:154:5)
    at node:internal/main/run_main_module:33:47
```

Create your own Error object

```
class ValidationError extends Error {
  constructor(message) {
    super(message)
    this.name = "ValidationError"
  }
}

function withdraw(amount) {
  if (amount <= 0) {
    throw new ValidationError("Amount must be
greater than zero")
  }
  console.log("Withdrawing:", amount)
}

try {
  withdraw(-10)
} catch (e) {
  console.log(e.name)
  console.log(e.message)
  console.log(e.stack)
}
```

```
//output
ValidationError
Amount must be greater than zero
ValidationError: Amount must be greater than zero
    at withdraw (C:\driveD\Working\2-
2568\INT141\SourceCodes\11\test.js:10:11)
    at Object.<anonymous> (C:\driveD\Working\2-
2568\INT141\SourceCodes\11\test.js:16:3)
    at Module._compile
(node:internal/modules/cjs/loader:1761:14)
    at Object..js (node:internal/modules/cjs/loader:1893:10)
    at Module.load (node:internal/modules/cjs/loader:1481:32)
    at Module._load (node:internal/modules/cjs/loader:1300:12)
    at TracingChannel.traceSync
(node:diagnostics_channel:328:14)
    at wrapModuleLoad (node:internal/modules/cjs/loader:245:24)
    at Module.executeUserEntryPoint [as runMain]
(node:internal/modules/run_main:154:5)
    at node:internal/main/run_main_module:33:47
```

throw Statement with not Error Object (not recommend)

No error name, message, and stack from your error string.

```
throw "Error2" // String type
throw 42 // Number type
throw true // Boolean type
throw { message: 'error' } // Object type
```

```
try {
  console.log('Start')
  throw 'Oops!'
} catch (e) {
  console.log(e.name)
  console.log(e.message)
  console.log(e.stack)
}
//output
Start
undefined
undefined
undefined
```

try…catch Example

The example calls a function that retrieves a month name from an array based on the value passed to the function. If the value does not correspond to a month number (1 – 12), an exception is thrown with the value 'Invalid month code' and the statements in the catch block set the monthName variable to 'unknown'.

```
function getMonthName(mo) {
  mo-- // Adjust month number for array index (so that 0 =
Jan, 11 = Dec)
  const months = [
    "Jan", "Feb", "Mar", "Apr", "May", "Jun",
    "Jul", "Aug", "Sep", "Oct", "Nov", "Dec",
  ]
  if (!months[mo]) {
    throw new Error("Invalid month code") // throw keyword
is used here
  }
  return months[mo]
}
```

```
function logMyErrors(e) {
  const timestamp = new Date().toISOString()
  if (e instanceof Error) {
    console.error(`[${timestamp}] ${e.name}: ${e.message}`)
    console.error(e.stack)
  } else {
    console.error(`[${timestamp}] Unknown error:`, e)
  }
}
```

```
try {
  monthName = getMonthName(19)
}
catch (e) {
  monthName = "unknown"
  logMyErrors(e)
}
```

```
[2026-05-03T11:02:56.555Z] Error: Invalid month code
Error: Invalid month code
  at getMonthName (C:\driveD\Working\2-2568\INT141\SourceCodes\11\test.js:18:11)
  at Object.<anonymous> (C:\driveD\Working\2-2568\INT141\SourceCodes\11\test.js:32:15)
  at Module._compile (node:internal/modules/cjs/loader:1761:14)
  at Object..js (node:internal/modules/cjs/loader:1893:10)
  at Module.load (node:internal/modules/cjs/loader:1481:32)
  at Module._load (node:internal/modules/cjs/loader:1300:12)
  at TracingChannel.traceSync (node:diagnostics_channel:328:14)
  at wrapModuleLoad (node:internal/modules/cjs/loader:245:24)
  at Module.executeUserEntryPoint [as runMain] (node:internal/modules/run_main:154:5)
  at node:internal/main/run_main_module:33:47
```

Golden Rules of Exception Handling

- Rule 1: **Throw Early -** Detect problems **as soon as they appear**.
- Rule 2: **Catch Late -** Do NOT catch errors in low-level modules unless you can fix them.

Application Example

❌ ตัวอย่างที่ catch เร็วเกินไป

```
function bankAPI() {
  throw new Error('Bank server offline')
}

function paymentService() {
  try {
    bankAPI()
  } catch (e) {
    console.log('Payment failed')
  }
}

function orderService() {
  paymentService()
  console.log('Order completed')
}

orderService()
```

```
//Output
Payment failed
Order completed
```

✅ ตัวอย่าง throw early, catch late

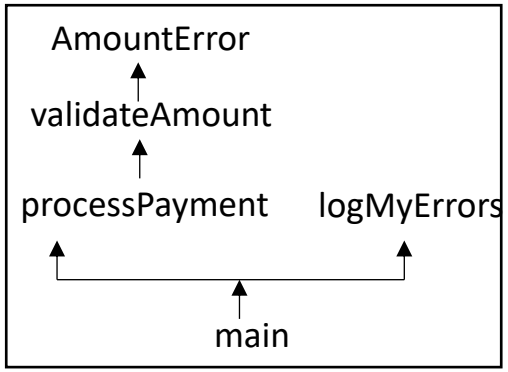
```
function bankAPI() {
  throw new Error('Bank server offline’)
}

function paymentService() {
  bankAPI()
}

function orderService() {
  try {
    paymentService()
    console.log('Order completed')
  } catch (e) {
    console.log('Cannot complete order')
    console.log('Reason:', e.message)
  }
}
orderService()
```

```
//Output
Cannot complete order
Reason: Bank server offline
```

Application Example




```
//AmountError.js
export class AmountError extends
Error {
  constructor(message) {
    super(message)
    this.name = 'AmountError'
  }
}
```

```
// payment.js
import { validateAmount } from './validation.js'
export function processPayment(amount) {
  validateAmount(amount)
  const taxRate = 0.07
  const total = amount + amount * taxRate
  return total
}
```

```
//validation.js  - throw where the problem is detected.
export function validateAmount(amount) {
  if (typeof amount !== 'number') {
    throw new AmountError('Amount must be a number')
  }
  if (amount <= 0) {
    throw new AmountError('Amount must be greater than zero')
  }
}
```

```
// main.js - catch where the decision can be made
import { processPayment } from './payment.js'
import { logMyErrors } from './logger.js'

function main() {
  try {
    const total = processPayment(-100)
    console.log('Payment total:', total)
  } catch (e) {
    console.log('Payment failed')
    logMyErrors(e)
  }
}


main()
```

```
//logger.js
export function logMyErrors(e) {
  if (e instanceof Error) {
    console.error('Error name:', e.name)
    console.error('Error message:', e.message)
    console.error('Stack:', e.stack)
  } else {
    console.error('Unknown error:', e)
  }
}
```